Technical Documentation

DevOps – Examination Report

Containerization, Kubernetes Deployment & Observability (Re-examination Assignment)

Version 1.0

written by

Ranga Lokesh Venkata Ganesh
ranga.lokesh1599072@gmail.com
Student ID: 10008311

Lecturer: Daniil Tashkan

Course: IT Security Management (ITSM) & DevOps

Date: 11 November 2026

Repository: https://github.com/lokesh-loki-o/crud-devops-project

Contents

1 Project Context and Objectives

2 Containerization and Setup

3 Kubernetes Deployment

4 Functional Validation and Load Testing

5 Observability and Fault Tolerance

6 Reflections and Challenges

7 Appendix (screenshots, configs, test results)

1 Project Context and Objectives

Acting as a DevOps consultant, the task was to take the CRUD web service built in Node.js during the lectures, rewrite it in Ruby on Rails, and onboard it into a containerized, scalable deployment on Kubernetes (Minikube), validated under load.

The four technical objectives were:

- **Containerization:** a production-grade, multi-stage Docker image and a Docker Compose stack with MySQL.
- **Kubernetes deployment:** declarative manifests with resource limits and health probes, verified on Minikube.
- **Load testing:** at least 500 concurrent virtual users for at least 60 seconds, with latency, error rate and throughput captured.
- **Observability and fault tolerance:** a simulated crash, self-healing, autoscaling and monitoring.

All work was done individually. Every number in this report comes from a real run; raw output is in the repository under `docs/evidence/` and `load-tests/results/`.

2 Containerization and Setup

2.1 Application

The application is a Rails 8.1 API-only app backed by MySQL 8.0 (official `mysql` image). It exposes full CRUD on tasks plus the Rails health endpoint `/up`, which Docker and Kubernetes use for health checks.

Method	Path	Purpose	Success status
GET	`/up`	Health check	200
GET	`/api/v1/tasks`	List tasks	200
GET	`/api/v1/tasks/:id`	Read one task	200 (404 if missing)
POST	`/api/v1/tasks`	Create a task	201 (422 if invalid)
PATCH / PUT	`/api/v1/tasks/:id`	Update a task	200 (422 if invalid)
DELETE	`/api/v1/tasks/:id`	Delete a task	200

A task has `title` (required), `description` and `completed` (default false). Request bodies use the form `{"task": {"title": "..."}}`.

2.2 Dockerfile

Following the lecturer's guidance to keep the Dockerfile simple (under 25 lines), it has 22 lines:

```
# Build stage: full official Ruby image compiles the gems (mysql2 needs build tools)
FROM ruby:3.3.10 AS build
WORKDIR /rails
ENV RAILS_ENV=production BUNDLE_WITHOUT=development:test
COPY Gemfile Gemfile.lock ./
RUN bundle install
COPY . .

# Runtime stage: slim official Ruby image, Puma running as a non-root user
FROM ruby:3.3.10-slim
WORKDIR /rails
ENV RAILS_ENV=production BUNDLE_WITHOUT=development:test
RUN apt-get update -qq && \
    apt-get install -y --no-install-recommends curl libmariadb3 && \
    rm -rf /var/lib/apt/lists/*
RUN useradd --create-home --uid 1000 rails
COPY --from=build /usr/local/bundle /usr/local/bundle
COPY --from=build --chown=rails:rails /rails /rails
USER rails
EXPOSE 3000
ENTRYPOINT ["/rails/bin/docker-entrypoint"]
CMD ["bundle", "exec", "puma", "-C", "config/puma.rb"]
```

- **Multi-stage build:** the full official `ruby:3.3.10` image installs and compiles the gems (the `mysql2` driver needs a compiler and MySQL headers); the official `ruby:3.3.10-slim` image runs the app, so build tools are not shipped.
- **Runtime packages:** only `libmariadb3` (needed by `mysql2`) and `curl` (needed by the Compose healthcheck).
- **Puma:** the server shipped with Rails, started by `CMD`. The entrypoint runs `rails db:prepare` and then `exec`s Puma, so Puma becomes PID 1 and receives signals directly.
- **Non-root:** the container runs as user `rails` (uid 1000), confirmed with `docker exec crud-devops-project id` → `uid=1000(rails)`.
- **Port:** `EXPOSE 3000`.
- **Image size:** 127 MB compressed content, 458 MB on disk.

2.3 Docker Compose

`docker-compose.yaml` defines two services on the named network `crud_net`:

- `crud-mysql`: the official `mysql:8.0` image, data on the named volume `mysql_data` mounted at `/var/lib/mysql`, with a `mysqladmin ping` healthcheck. Its port is published only on 127.0.0.1.
- `web`: built from the Dockerfile, with a healthcheck that probes `http://localhost:3000/up` every 10 s. It starts only after MySQL reports healthy (`depends_on: condition: service_healthy`).

Validation: both containers reported Healthy, and all 10 functional checks passed (list, create, read, PATCH, PUT, persistence of the update, validation error 422, delete, 404 after delete, and `/up`). A row also survived `docker compose down` and `up` with the volume kept, which confirms persistence. Output is in Appendix A and B.

2.4 Assumptions

- **Original endpoints:** the original Node.js source was not available while rewriting. It is assumed to be the lecture's task CRUD API, so the Rails app implements the five standard REST operations on one resource with JSON bodies and conventional status codes (201 on create, 404 for a missing record, 422 on invalid input). If the original used different route names, only `config/routes.rb` would change.
- **Versions:** Ruby 3.3.10, Rails 8.1.4 (a currently supported release), Puma 8.0.2, MySQL 8.0.
- **Ports:** the app listens on 3000 and MySQL on 3306.
- **Database configuration:** one production database, `crud_devops_project_production`, accessed by a dedicated user `crud_app` rather than root. Rails' optional cache, queue and cable databases are not used and were removed.
- **Migrations:** `rails db:prepare` runs on container start. This is safe with one replica; with several replicas starting at the same time, a dedicated migration Job would be better.
- **Environment variables:** `DB_HOST`, `DB_USERNAME`, `DB_PASSWORD`, `RAILS_MASTER_KEY`, `RAILS_ENV`, `PORT`, `WEB_CONCURRENCY`, `RAILS_MAX_THREADS`. Secrets live in a git-ignored `.env`; `env.example` documents them.
- **Dockerfile simplification:** the load tests in section 4 were run with an earlier, longer version of the image that also used the jemalloc memory allocator. The simplified 22-line image was rebuilt afterwards and passed the same 10 functional checks on Docker Compose and was rolled out on Kubernetes (Appendix B). Memory figures in section 4 may therefore differ slightly from the final image.
- **TLS:** not terminated by the app, because it is only reached inside the cluster for this assignment.

3 Kubernetes Deployment

3.1 Manifests

All manifests are in `k8s/`:

File	Contents
`namespace.yaml`	Namespace `crud-app`
`configmap.yaml`	Non-secret settings (Rails env, DB host, Puma workers and threads)
`secret.example.yaml`	Shape of the Secret only; the real Secret is created from `.env` and never committed
`mysql.yaml`	MySQL StatefulSet with a 2 Gi PersistentVolumeClaim and a headless Service
`deployment.yaml`	The Rails Deployment
`service.yaml`	ClusterIP Service for the app
`hpa.yaml`	Horizontal Pod Autoscaler (section 5.2)

`deployment.yaml` uses `apiVersion: apps/v1` and `kind: Deployment`, with name `crud-web`, namespace `crud-app`, label `app: crud-web`, `spec.replicas: 1`, and `selector.matchLabels` matching the pod template labels. The pod runs as uid 1000 (`runAsNonRoot: true`) with privilege escalation disabled.

Container	CPU request / limit	Memory request / limit
web (Rails)	500m / 2	512Mi / 1Gi
mysql	250m / 1	512Mi / 1Gi
wait-for-mysql (init)	50m / 100m	64Mi / 128Mi

All three probes on the web container use HTTP GET `/up`, a real health endpoint rather than a TCP check:

Probe	Period	Failure threshold	Purpose
startupProbe	5 s	30 (up to 150 s)	Gives `db:prepare` and boot time before liveness applies
readinessProbe	5 s	3	Removes an unready pod from the Service so it gets no traffic
livenessProbe	10 s	6	Restarts a hung process automatically

The liveness threshold is deliberately tolerant so a pod that is only slow under load is not restarted. `/up` confirms the app booted but does not query MySQL, so a database outage does not restart the web pods. An init container waits for MySQL's port so the app does not crash-loop on first start.

3.2 Service type

`k8s/service.yaml` uses **ClusterIP** (port 80 → container port 3000). The assignment asks for exposure inside the cluster, and ClusterIP gives a stable internal address with the smallest attack surface. **NodePort** would open a port on the node with no need, and **LoadBalancer** needs a cloud provider (or `minikube tunnel`) to get an external IP. For debugging from the host, `kubectl port-forward svc/crud-web 8080:80` is used instead.

3.3 Deployment on Minikube

The image was loaded with `minikube image load`, the Secret was created from `.env`, and the manifests were applied. Verification:

```
$ kubectl -n crud-app get deploy,statefulset,pods,svc,pvc
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/crud-web     1/1     1            1           79s

NAME                        READY   AGE
statefulset.apps/mysql      1/1     79s

NAME                                READY   STATUS    RESTARTS   AGE
pod/crud-web-75bb9db5d4-g74kt       1/1     Running   0          79s
pod/mysql-0                         1/1     Running   0          79s

NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/crud-web    ClusterIP   10.107.234.11   <none>        80/TCP     79s
service/mysql       ClusterIP   None            <none>        3306/TCP   79s

NAME                                          STATUS   CAPACITY   ACCESS MODES
persistentvolumeclaim/mysql-data-mysql-0      Bound    2Gi        RWO

$ kubectl -n crud-app get endpointslices
NAME              ADDRESSTYPE   PORTS   ENDPOINTS    AGE
crud-web-5mz65    IPv4          3000    10.244.0.4   79s
mysql-k2p7g       IPv4          3306    10.244.0.5   79s
```

The Deployment is available (1/1), the Service has the pod as its endpoint, and the PVC is bound. All 10 functional checks also passed through the Service (`http://crud-web`). After the Dockerfile was simplified, the new image was loaded and rolled out; the new pod reached 1/1 Running (Appendix B).

3.4 Assumptions

- Minikube with the Docker driver, 6 CPUs and 6 GiB memory, Kubernetes v1.37.0, on an Apple Silicon (arm64) Mac, with the metrics-server addon.
- The image is loaded into Minikube directly, so no registry is used (`imagePullPolicy: IfNotPresent`).
- MySQL runs in the cluster as a StatefulSet with one replica; high availability for the database is out of scope.

4 Functional Validation and Load Testing

4.1 Functional validation

`load-tests/functional-test.sh` checks every endpoint and status code (10 checks). It passed on Docker Compose, through the Kubernetes Service, after the crash test, and with the final simplified image.

4.2 Load test scenario

The tool is **k6** (`load-tests/k6-load-test.js`). It ramps to **500 virtual users in 15 s, holds 500 for 60 s**, then ramps down over 10 s. Each iteration runs the full cycle list → create → read → update → delete and sleeps 1 s. Deleting each record keeps the table small, so the cost of the list call stays constant. It captures latency (p50/p95/p99/max), error rate and throughput.

k6 runs **inside the cluster** as a pod (`load-tests/k6-runner.yaml`, started by `load-tests/run-in-cluster.sh`) and targets the Service, so results measure the application rather than a macOS port-forward tunnel. It shares the node's 6 CPUs with the application.

Puma is configured with `WEB_CONCURRENCY=2` worker processes and `RAILS_MAX_THREADS=5` threads each, so one pod serves **10 requests at once**. The database pool is one connection per thread.

4.3 Results

Run	Replicas	Requests	Avg req/s	Error %	p50 ms	p95 ms	p99 ms	Max ms
1 baseline	1	80,120	933.5	0.00	257	594	782	1,370
2 repeat + counters	1	77,850	905.2	0.00	270	632	920	1,360
3 HPA, 180 s hold	1 → 3	170,205	826.4	0.00	333	873	1,190	1,950
4 fixed replicas	3	103,500	1,207.3	0.00	102	481	777	1,230

Table 1: Summary of all load-test runs (averages include ramp-up and ramp-down)

Window (s)	Req/s	Error %	p50 ms	p95 ms	p99 ms
0–10	867.0	0.00	2	31	85
10–20	932.3	0.00	311	540	606
20–30	921.2	0.00	316	642	754
30–40	941.9	0.00	297	749	1,224
40–50	1,086.1	0.00	255	449	677
50–60	934.3	0.00	282	691	1,327
60–70	1,010.4	0.00	281	594	727
70–80	1,026.8	0.00	265	451	757
80–90	292.0	0.00	5	119	155

Table 2: Run 1 (baseline, 1 replica) – latency percentiles and error rate over time in 10-second windows

The same over-time table for every run is in `load-tests/results/<run>/analysis.md`.

4.4 CPU and memory per pod

Pod	CPU	Memory peak	Memory limit
crud-web	1.33 cores average over the run (cgroup); 1.56 cores peak (kubectl top)	384 MiB	1 GiB
mysql-0	297m peak (kubectl top)	382 MiB	1 GiB

Table 3: Resource use during run 2 (one replica)

`kubectl top` refreshes slowly, so its values are coarse. The web pod's CPU average was therefore taken from `usage_usec` in its cgroup, which is exact. The CPU limit was never hit: `nr_throttled` stayed at 0.

4.5 Analysis

The load was handled satisfactorily. Every run had 0% errors, and with one replica p95 stayed around 600 ms, well inside the 2-second target set as a k6 threshold.

Relation to Puma. At about 930 req/s and about 258 ms average latency, Little's Law gives roughly 240 requests in the system at once, against only 10 Puma threads. Most of each request's time was spent waiting in the queue. This matches the jump from about 2 ms p50 at low load (first 10 s) to about 300 ms at full load.

CPU-bound, not memory-bound. Memory peaked at about 38% of its limit. Because of Ruby's Global VM Lock, each Puma worker can use about one core, so 2 workers cap near 2 cores; the pod reached up to 1.56 cores. Adding workers by running 3 replicas (run 4) lifted throughput from about 950 to 1,600–1,650 req/s for the first part of the run and cut p95 by about 20%, which supports this. MySQL was not saturated (576m peak of 1 core in run 4).

Limits of this conclusion. Run 4 did not sustain its peak: throughput began to fall about 40 s in and was down to 880–970 req/s by the end. The cause was not isolated. All pods and k6 share one 6-CPU node, so it could be CPU contention on the node or a growing MySQL cost. More runs varying only `WEB_CONCURRENCY` would be needed to confirm it.

5 Observability and Fault Tolerance

5.1 Crash simulation

The Puma master process was killed with SIGKILL from the Minikube node while the pod was serving:

```
$ kubectl -n crud-app get pods   (before)
crud-web-75bb9db5d4-g74kt   1/1   Running   0            5m18s

$ minikube ssh "sudo pkill -9 -f '[p]uma 8.0.2'"     # 04:45:11, SIGKILL Puma

$ kubectl -n crud-app get pods   (polled every 3 s)
04:45:12  crud-web-75bb9db5d4-g74kt   1/1   Running   0             5m19s
04:45:15  crud-web-75bb9db5d4-g74kt   0/1   Running   1 (3s ago)    5m22s
04:45:18  crud-web-75bb9db5d4-g74kt   0/1   Running   1 (6s ago)    5m25s
04:45:21  crud-web-75bb9db5d4-g74kt   1/1   Running   1 (9s ago)    5m28s

restartCount=1
lastState: terminated, exitCode 137, reason Error

Warning  Unhealthy  Startup probe failed: Get "http://10.244.0.4:3000/up":
                    connect: connection refused
```

- RESTARTS went from **0 to 1**, and the previous container exited with code **137** (SIGKILL).
- The pod went to **0/1**, so the readiness probe took it out of the Service, and was **1/1** again about **9 seconds** later.
- The events show the startup probe failing with "connection refused" while Puma restarted.

Deleting the pod also worked: the ReplicaSet created a replacement that was Ready within seconds, and all 10 functional checks passed afterwards.

5.2 Horizontal Pod Autoscaler

`k8s/hpa.yaml` (`autoscaling/v2`) scales `crud-web` between **1 and 3 replicas** at **70% of the CPU request**, i.e. above 350m average per pod. It is applied separately, so the 1-replica baseline runs without it. Output during run 3:

```
TIME      TARGETS          MINPODS  MAXPODS  REPLICAS
04:47:42  cpu: <unknown>/70%   1        3        1
04:47:52  cpu: 302%/70%        1        3        1
04:48:02  cpu: 302%/70%        1        3        3
04:48:52  cpu: 310%/70%        1        3        3
04:49:52  cpu: 103%/70%        1        3        3
```

The HPA read 302% CPU and scaled to 3 replicas, but **throughput did not improve**. The two new pods stayed near 50m CPU. The likely reason is that k6 keeps its HTTP connections open, and a ClusterIP Service balances per connection, so existing connections stayed on the first pod. Run 4 supports this: with 3 replicas running before the test started, connections spread out and the extra pods did real work (about 1,150–1,210m each).

Autoscaling a workload with long-lived connections therefore needs connection limits or Layer 7 load balancing to rebalance traffic. The CPU metric also took about 80 seconds to become available after the HPA was applied, which is too slow for short bursts.

5.3 Monitoring

- **kube-prometheus-stack** (Prometheus, Grafana, Alertmanager, kube-state-metrics and node-exporter) was installed with Helm in the `monitoring` namespace. Its ServiceMonitors scrape the kubelet, kube-state-metrics and node-exporter, so per-pod CPU and memory in `crud-app` can be followed in Grafana's Kubernetes dashboards during a load test.
- **kubectl top and events:** `kubectl top pod` was sampled every 5 s during each run, and pod events and restart counts showed probe failures and restarts.
- **cgroup counters:** because `kubectl top` is too coarse for short peaks, CPU (`usage_usec`), throttling (`nr_throttled`) and peak memory were read directly from the web container's cgroup before and after each run.

6 Reflections and Challenges

What worked. The image, Compose stack and Kubernetes deployment worked as intended. The application handled 500 concurrent users with 0% errors in every run, and Kubernetes restarted the crashed container on its own in about 9 seconds without manual action. Running k6 inside the cluster was a good decision, because results measured from the Mac through a port-forward would mainly have measured the tunnel.

Problems I ran into.

- **The `/up` route was missing at first.** The health checks and probes target `/up`, but the route had not been defined in the API app, so the checks failed until I added it to `config/routes.rb`.
- **A local development server was already using port 3000.** Requests I thought were reaching the container were going to the local server, which made early results confusing until I found and stopped it.
- **The HPA scaled up but did not help.** It reached 3 replicas, but throughput stayed the same because k6's open connections stayed on the first pod. I learned that autoscaling alone does not spread load when connections are long-lived.

- **Run 4 did not hold its peak.** With 3 fixed replicas, throughput rose at first and then fell. I could not isolate the cause, since everything shared one node.
- **The Dockerfile was too long.** My first version was about 60 lines. Following the lecturer's guidance, I reduced it to 22 lines and checked that it still meets every requirement and passes the functional tests.
- **The original Node.js code was not available,** so the endpoints are an assumption (section 2.4).

What I would change. I would run the database migration as a separate Kubernetes Job instead of on container start, test with separate nodes for the load generator and the application, and repeat the 3-replica test while varying only the number of Puma workers. For autoscaling I would add Layer 7 load balancing (an Ingress) so new pods receive traffic.

7 Appendix

A. Repository structure

```
crud-devops-project/
├── app/, config/, db/          Rails application
├── Dockerfile                  multi-stage image (22 lines)
├── docker-compose.yaml         app + MySQL, network crud_net, volume mysql_data
├── env.example                 required environment variables
├── k8s/                        namespace, configmap, secret.example, mysql,
│                               deployment, service, hpa
├── load-tests/                 functional-test.sh, k6-load-test.js,
│   └── results/<run>/          k6-runner.yaml, run-in-cluster.sh, analyze.py
├── docs/evidence/              captured command output
├── frontend/                   optional React client (not in the API image)
└── README.md
```

B. Final image: build, functional test and rollout

Docker Compose rebuilt with the simplified Dockerfile, both containers healthy, non-root user, 10/10 functional checks passed, image size, and the rollout of the new image on Minikube:

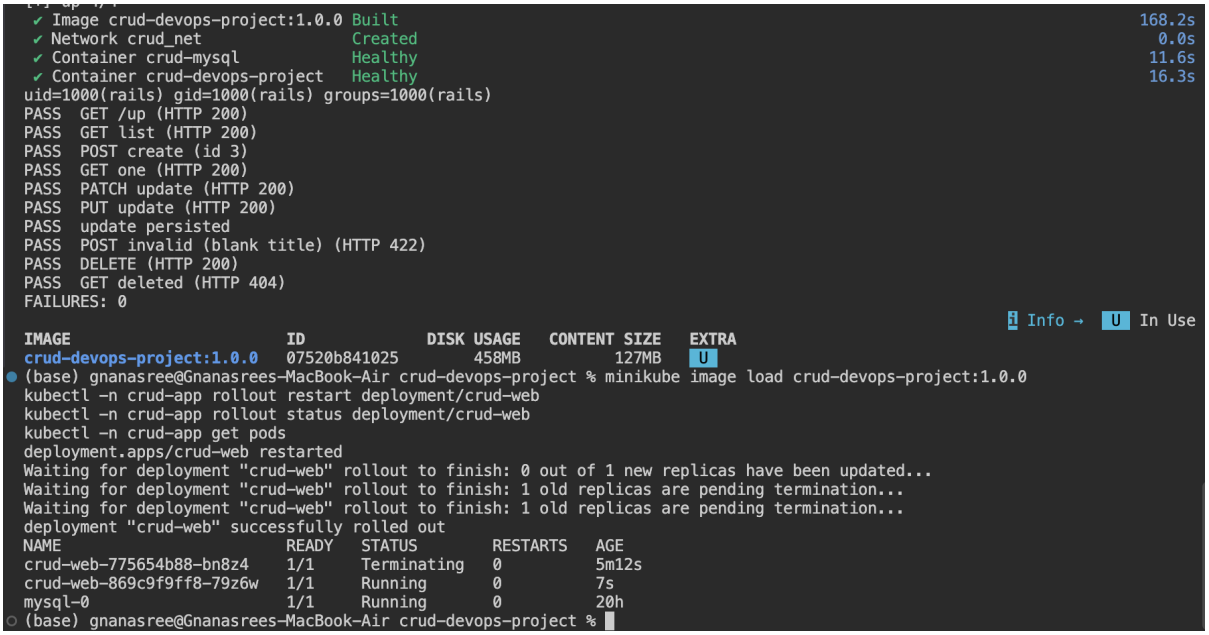


Figure 1: Rebuild and verification of the final 22-line image

C. Raw results

- `docs/evidence/01-compose.txt`: Compose status, functional test, non-root check, persistence test
- `docs/evidence/02-kubernetes.txt`: `kubectl get`, Deployment description, endpoint slices, functional test through the Service
- `docs/evidence/03-fault-tolerance.txt`: crash simulation, restart count, events, pod deletion
- `docs/evidence/04-hpa.txt`: HPA scaling during run 3
- `load-tests/results/<run>/`: k6 output, `summary.json`, `analysis.md`, `kubectl top` samples and cgroup counters for runs 1–4